

# InBank — Bank Management System

---

Interview preparation guide · Author: Samiksha · GitHub:

[github.com/samiksha1803/Banking-system](https://github.com/samiksha1803/Banking-system) · Stack: Java 21, Servlets, JSP, Hibernate/JPA, PostgreSQL, Tomcat 9, Maven

## 30-second elevator pitch

InBank is a Java web banking application I built using Servlets and JSP on Apache Tomcat, with Hibernate and JPA for persistence on PostgreSQL. Users can register, log in, deposit, withdraw, and transfer money to other customers using a 12-digit account number. Every money movement is recorded in a transaction ledger with reference numbers and timestamps. There is an admin module for customer and ledger management, plus session-based security and password hashing.

## 1. Why this project (for HR / fresher rounds)

- Demonstrates **end-to-end web development** in Java (not only console programs).
- Shows **database design**, ORM, and **transaction integrity** (transfers update two balances atomically).
- Includes **authentication**, role-based admin access, and validation.
- Deployable as a **WAR** on Tomcat — similar to industry Java EE style apps.

## 2. Technology stack (what to say when asked)

| Layer    | Technology     | Your one-line explanation                     |
|----------|----------------|-----------------------------------------------|
| Language | Java 21        | Core business logic, servlets, services.      |
| Web      | Servlets + JSP | Servlets handle HTTP; JSP renders HTML views. |

|          |                     |                                                            |
|----------|---------------------|------------------------------------------------------------|
| ORM      | Hibernate 5.6 + JPA | Maps Java entities to PostgreSQL tables; JPQL queries.     |
| Database | PostgreSQL          | Relational storage for users, transactions, notifications. |
| Build    | Maven (WAR)         | Dependencies and packaging for Tomcat deployment.          |
| Server   | Apache Tomcat 9     | Hosts the application at /InBank.                          |
| Tools    | Lombok, pgAdmin     | Reduces boilerplate on entities; DB administration.        |

### 3. Architecture (layered design)

Interviewers like hearing you organized the code in layers:

```

Browser (JSP + CSS)
| HTTP GET/POST
Controller layer -> controller.*Servlet (Login, Transfer, Admin, ...)
|
Service layer    -> BankService, AssistantService (business rules)
|
DAO layer       -> UserDao, TransactionDao, NotificationDao (queries)
|
JPA / Hibernate -> EntityManager, entities in dto package
|
PostgreSQL      -> tables: users, transactions, notifications

```

#### Filters & listener:

- `EncodingFilter` — UTF-8 for requests/responses.
- `AuthFilter` — protects dashboard, transfer, admin URLs; refreshes user from DB each request.
- `AppListener` — on startup: DB bootstrap, default admin user, legacy data fixes.
- `web.xml` — registers all servlets and filters ( `metadata-complete=true` ).

## 4. Database entities (tables)

### User ( users )

- `id`, `username`, `password` (hashed), `fullName`, `email`, `phone`
- `accountNumber` — unique 12-digit number (starts with 6210)
- `balance` — `BigDecimal` for money precision
- `role` — USER or ADMIN; `active` — account enabled/disabled

### Transaction ( transactions )

- Ledger row per money movement: `type` (DEPOSIT, WITHDRAW, TRANSFER\_IN, TRANSFER\_OUT, adjustments)
- `amount`, `balanceAfter`, `referenceNumber`, `createdAt`
- `note`, counterparty name/account for transfers
- Linked to `user_id`

### Notification ( notifications )

- In-app alerts after deposits, transfers, password changes, etc.
- Optional email via `mail.properties` (SMTP).

Hibernate setting: `hbm2ddl.auto=update` — schema evolves on startup (good for learning; production would use migrations).

## 5. Main features (map to your resume bullets)

| Feature              | How it works (short)                                                                             |
|----------------------|--------------------------------------------------------------------------------------------------|
| Registration         | Validates username/email; hashes password; generates account number; welcome notification.       |
| Login / Logout       | Session stores <code>User</code> (password cleared); 20 min timeout; logout invalidates session. |
| Dashboard            | Balance, deposit/withdraw forms, recent transactions.                                            |
| Deposit / Withdraw   | <code>BankService</code> updates balance + inserts transaction in one DB transaction.            |
| Insufficient balance | Checked before withdraw/transfer; user sees error message.                                       |
| Transfer             | By recipient account number; two ledger rows, same reference; pessimistic locks on both users.   |
| Transaction history  | JPQL search by type, date range, reference, name.                                                |
| Profile / Password   | Update contact info; PBKDF2 password change with validation.                                     |
| Admin                | Stats, customer search, activate/deactivate, manual credit/debit, full ledger.                   |
| Assistant            | Rule-based answers from user's balance/history (no external AI API).                             |

## 6. Request flow examples (explain in depth if asked)

### Login

1. POST `/login` -> `LoginServlet`
2. `BankService.login` — JPQL find user, `PasswordUtil.matches`

3. Legacy plain passwords upgraded to hash on successful login
4. Session attribute `user` set; redirect to dashboard or admin

### Transfer (most impressive to explain)

1. POST `/transfer` — `AuthFilter` ensures logged-in user
2. `BankService.transfer` inside single `EntityTransaction`:
  - Validate amount and 12-digit account number
  - Find sender and receiver; reject self-transfer and inactive accounts
  - **Pessimistic write locks** on both users (lower user id first — avoids deadlock)
  - Subtract from sender, add to receiver; two `Transaction` rows with same reference
  - Notifications for both parties
3. On success: refresh session user, flash message, redirect to dashboard

**Honest note for interviews:** User self-service *Deposit* is simplified for demos. Real banks usually credit accounts via branch, ATM, or payment rails; admin adjustment models that idea.

## 7. Security (what you implemented)

- **Passwords:** PBKDF2 with salt (not stored in plain text for new users).
- **Session:** `HttpOnly` cookie; user object in session; auth filter on protected routes.
- **Admin routes:** Only `role=ADMIN` can access `/admin/*`.
- **Validation:** Central classes — username, email, phone, amounts, account numbers.
- **SQL injection:** JPA parameterized queries (JPQL `:param`), not string-concat SQL.

## 8. Project structure (folders to remember)

| Package / folder                      | Responsibility                                 |
|---------------------------------------|------------------------------------------------|
| <code>controller/</code>              | HTTP endpoints (servlets)                      |
| <code>service/BankService.java</code> | Core banking rules and transactions            |
| <code>dao/</code>                     | Database queries                               |
| <code>dto/</code>                     | JPA entities (User, Transaction, Notification) |
| <code>filter/</code>                  | Auth, encoding                                 |
| <code>util/</code>                    | Money, PasswordUtil, Validators, Web helpers   |
| <code>webapp/WEB-INF/views/</code>    | JSP pages (not directly URL-accessible)        |
| <code>persistence.xml</code>          | JDBC URL, dialect, Hibernate settings          |

## 9. Live demo script (5 minutes in interview)

1. Open `http://localhost:8080/InBank/`
2. Register User A and User B — show account numbers on Profile
3. Login as A -> Deposit -> show balance and Transactions (reference + timestamp)
4. Transfer to B's account number -> success message -> login as B -> balance increased
5. Optional: Login as `admin` / `admin123` -> Admin stats and ledger

## 10. Common interview questions & sample answers

**Why Servlets and not Spring Boot?**

I wanted to learn the Java web foundation: HTTP, sessions, filters, and deployment on Tomcat. Spring Boot adds convenience later; this project shows I understand the underlying servlet model and JPA.

### **Why Hibernate / JPA?**

It maps my Java classes to tables, generates SQL, and gives me transactions and locking. I use JPQL for readable queries and keep SQL out of servlets.

### **Why PostgreSQL?**

Reliable open-source RDBMS; good for relational banking data (users, ledger). I used pgAdmin to inspect tables and verify transfers.

### **How do you prevent double-spending or wrong balances on transfer?**

All updates happen in one database transaction. I use pessimistic locks on both user rows, check balance before debit, and commit or rollback together. Two transaction records share one reference number.

### **What is the difference between session and database transaction?**

HTTP session stores logged-in user state in the server. A JPA `EntityTransaction` is a short DB unit of work (begin/commit) for one operation like a transfer.

### **What challenges did you face?**

Servlet mapping with web.xml, session expiry during transfer (fixed with clear messages and session refresh), character encoding for minus signs in JSP, and deploying from Eclipse to Tomcat with the correct context path /InBank.

### **What would you improve next?**

Spring Boot or REST API layer, Flyway migrations instead of hbm2ddl auto-update, admin-only deposits or deposit approval workflow, unit tests for BankService, and HTTPS in production.

## **11. Keywords to use confidently**

MVC-style layering · WAR deployment · JPA entities · JPQL · ACID transaction · Pessimistic locking · Session management · Filter chain · DTO/DAO pattern · BigDecimal for currency · Role-based access · Ledger / audit trail

## 12. Setup reminder (if they ask “can you run it?”)

1. PostgreSQL database `inbank` ; configure `persistence.xml`
2. Import Maven project in Eclipse; add to Tomcat 9; Maven Update; Start server
3. URL: `/InBank/` — not Tomcat root unless configured

InBank Interview Guide · For personal preparation · [github.com/samiksha1803/Banking-system](https://github.com/samiksha1803/Banking-system)